



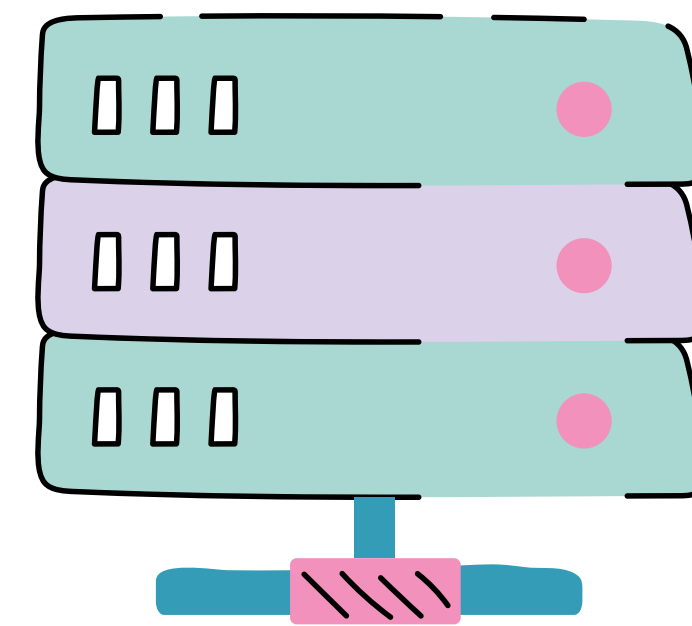
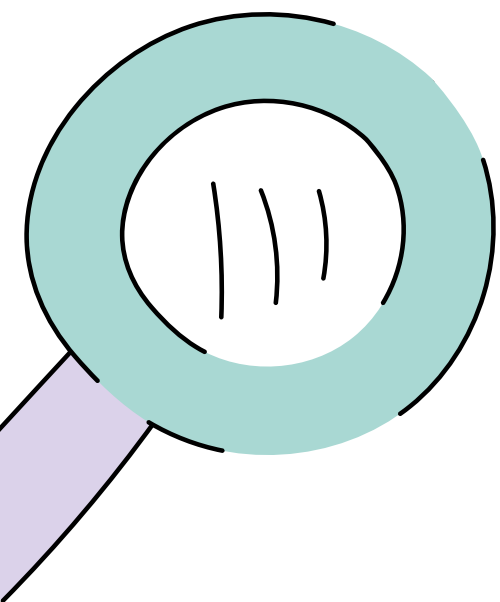
ESTRUTURA DE DADOS: TIPO ABSTRATO DE DADOS

Dra. Maíla de Lima Claro
maila.claro@ifpi.edu.br



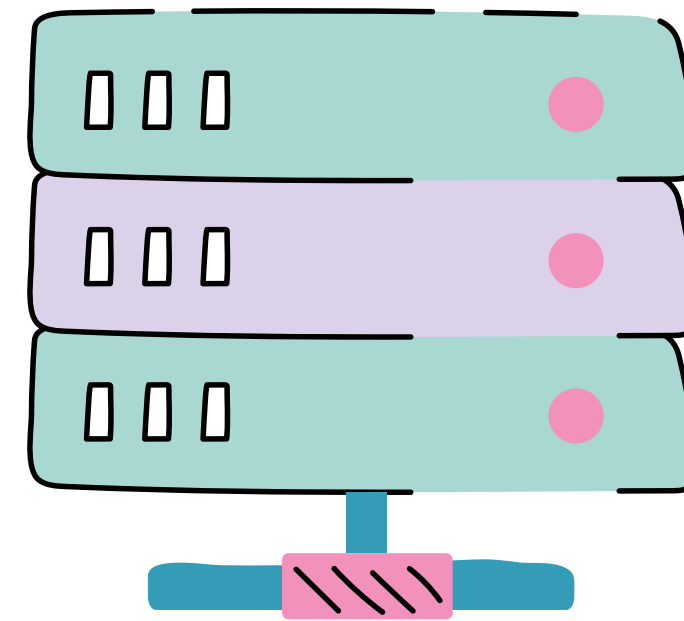
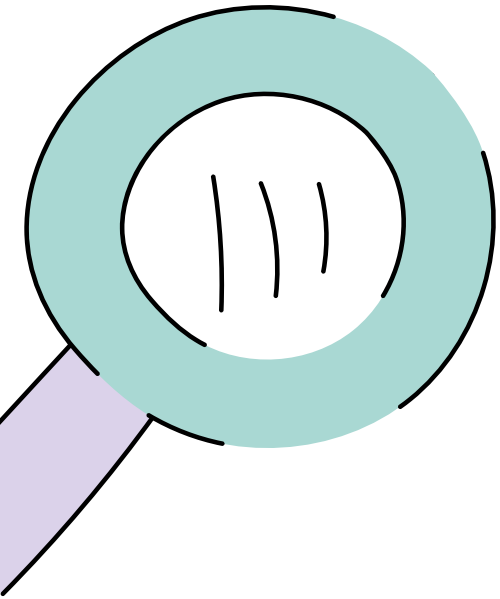
Tipo Abstrato de Dados (TADs)

- As variáveis estudadas até o momento podem armazenar somente dados simples. Ex: int, float, char;
- Contudo, muitas vezes precisamos armazenar dados compostos, ou seja, várias variáveis que se referem ao mesmo conceito do mundo real.
- Exemplos:
 - Uma pessoa que possui nome, idade, peso e altura.
 - Um aluno que possui nome, número de matrícula e curso.
 - Um livro que possui título, gênero, autor(es), ISBN, etc.
 - Um jogo que possui nome, gênero, valor, empresa desenvolvedora.



Tipo Abstrato de Dados (TADs)

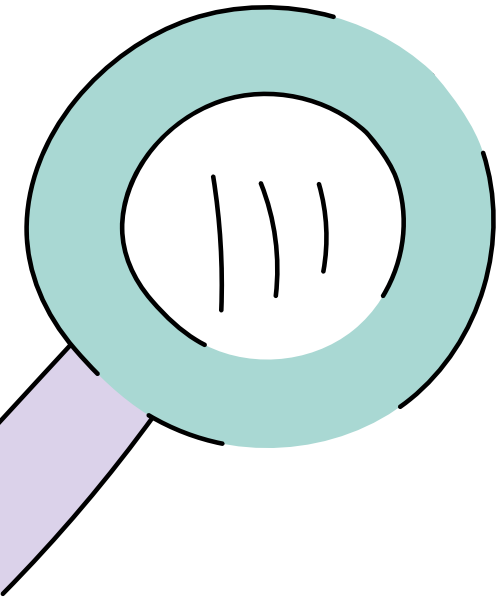
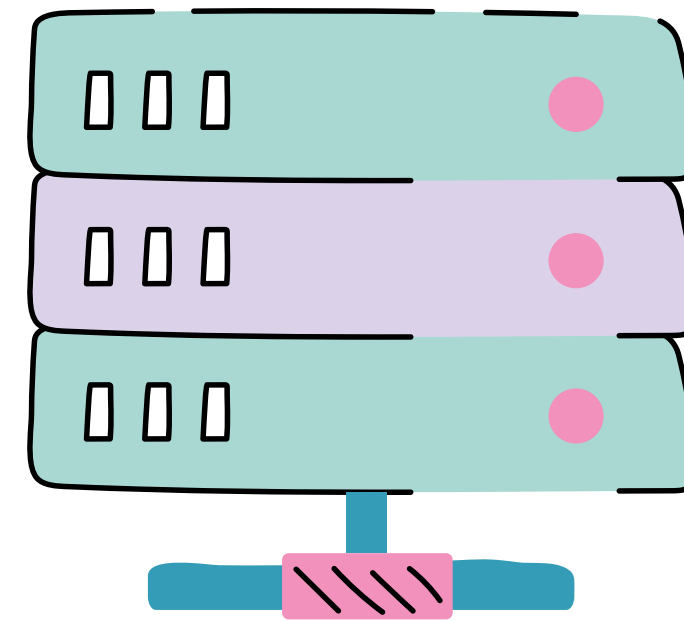
- Se pensarmos no mundo real, não temos conjuntos de informações separadas, elas fazem parte de um todo.
- Desta forma seria melhor termos um “vetor de todos” ao invés de vetores separados de informações.
- Isso é possível utilizando as estruturas de dados.



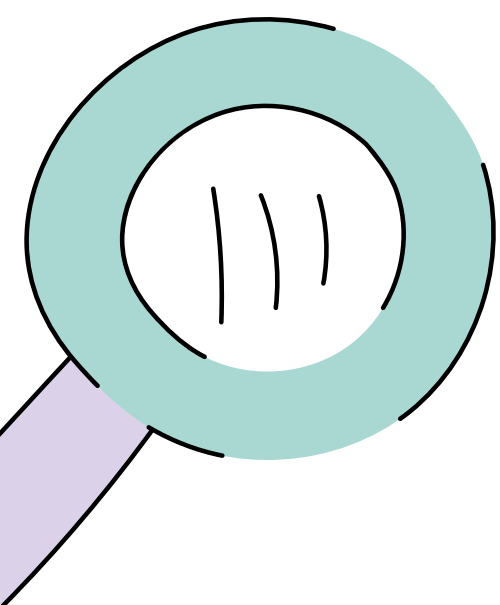
Registros e Estruturas

- Uma estrutura agrupa várias variáveis numa só. Assim, ela serve para agrupar um conjunto de dados não similares (que não são do mesmo tipo), formando um novo tipo de dados.

```
struct stEndereco
{
    char Rua[50];
    int numero;
    char bairro[20];
    char cidade[20];
    char siglaEstado[2];
};
```

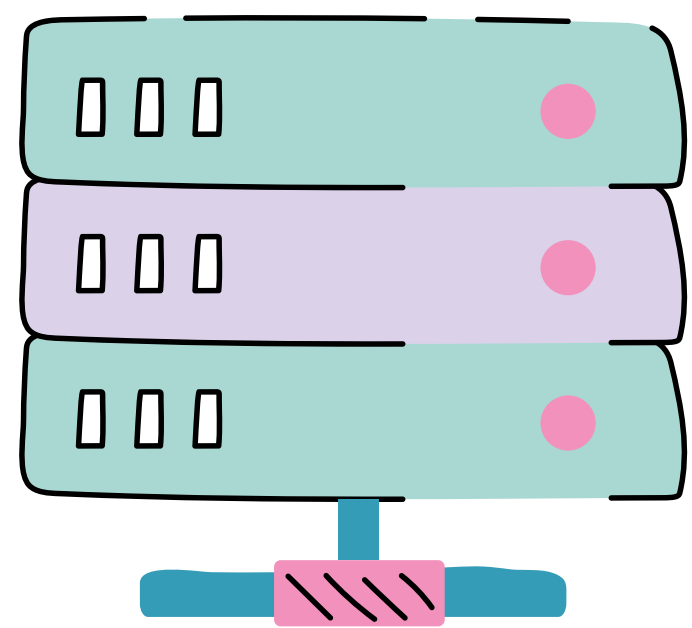


Sintaxe – Estrutura de Dados



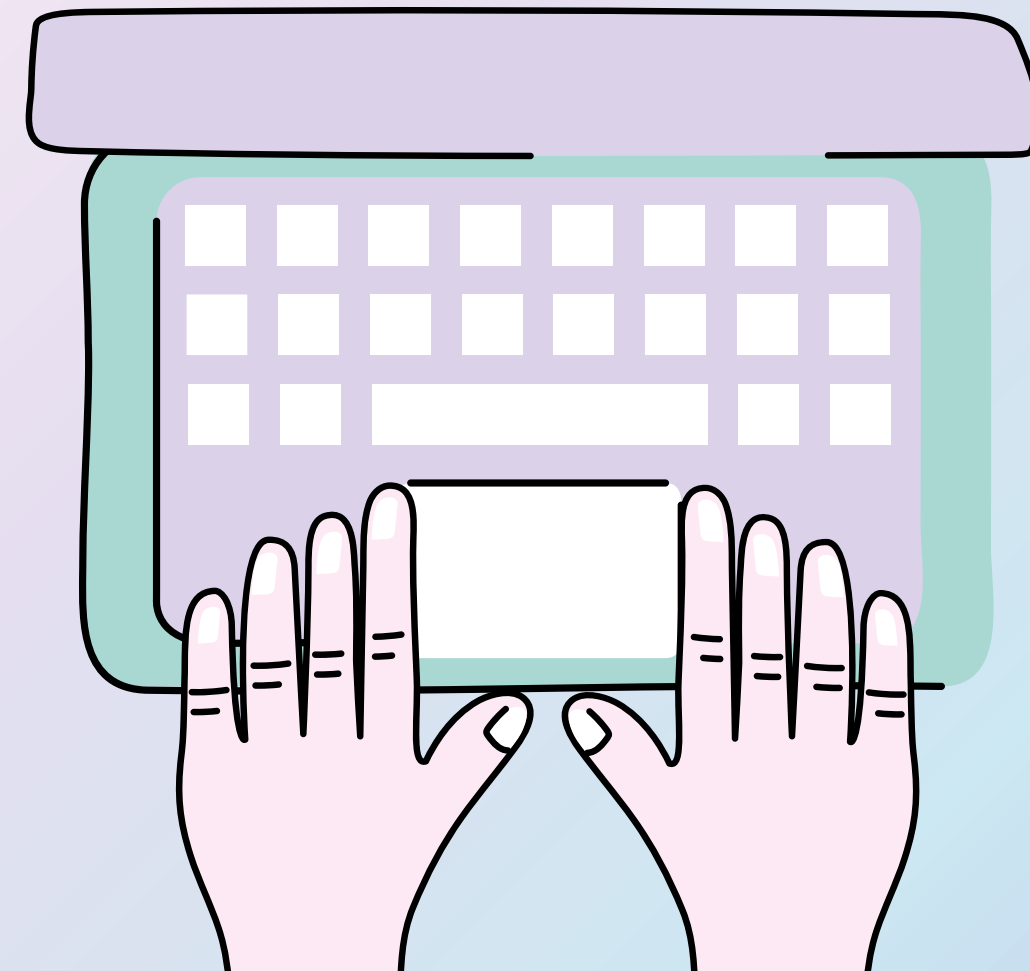
```
struct <nome> {  
    <declaração de variáveis>  
    ...  
    <declaração de variáveis>  
};  
<declaração de variáveis> =  
    <tipo> <nome>, <nome>,  
    ...;
```

```
struct paciente {  
    char nome[20];  
    int idade;  
    float altura, peso;  
};  
struct ponto {  
    int x,y;  
};
```



Declarando e Manipulando uma Estrutura

```
struct paciente{  
    char nome[20];  
    int idade;  
    float altura, peso;  
};  
  
int main(){  
    struct paciente p;  
    strcpy(p.nome, "Joao");  
    p.idade = 28;  
    p.altura = 1.91;  
    p.peso = 88.0;  
}
```

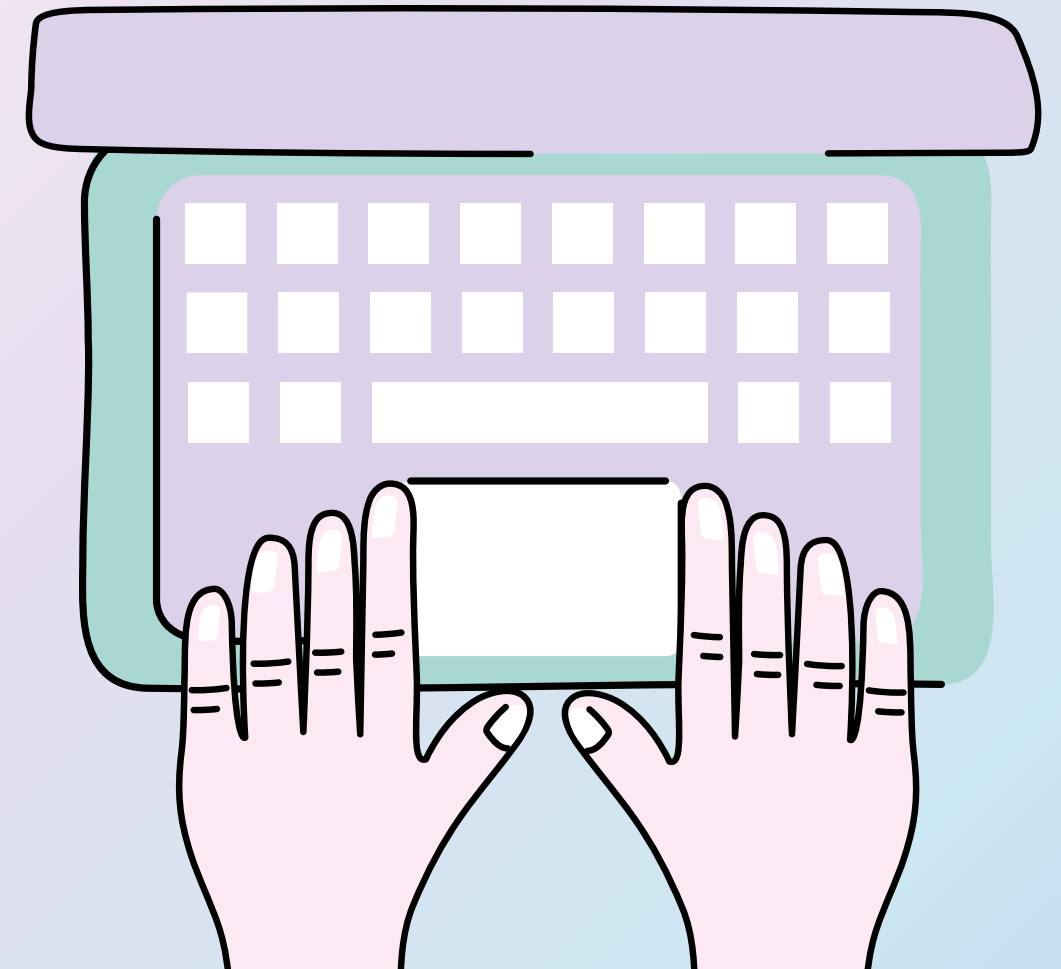


TYPDEF

```
struct paciente{  
    char nome[20];  
    int idade;  
    float altura, peso;  
};
```

```
typedef struct paciente Paciente;
```

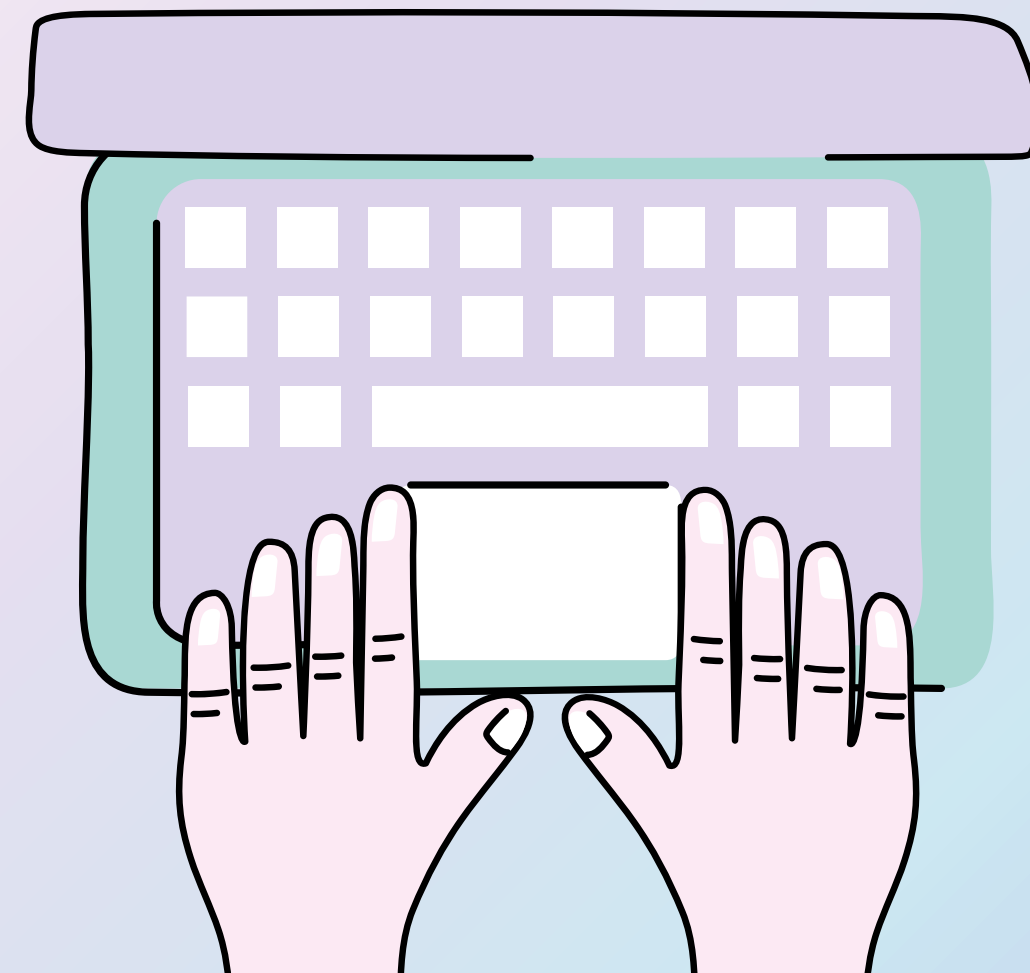
```
int main() {  
    Paciente p;  
    strcpy(p.nome, "Joao");  
    p.idade = 28;  
    p.altura = 1.91;  
    p.peso = 88.0;  
}
```



Passando uma Estrutura por Parâmetro

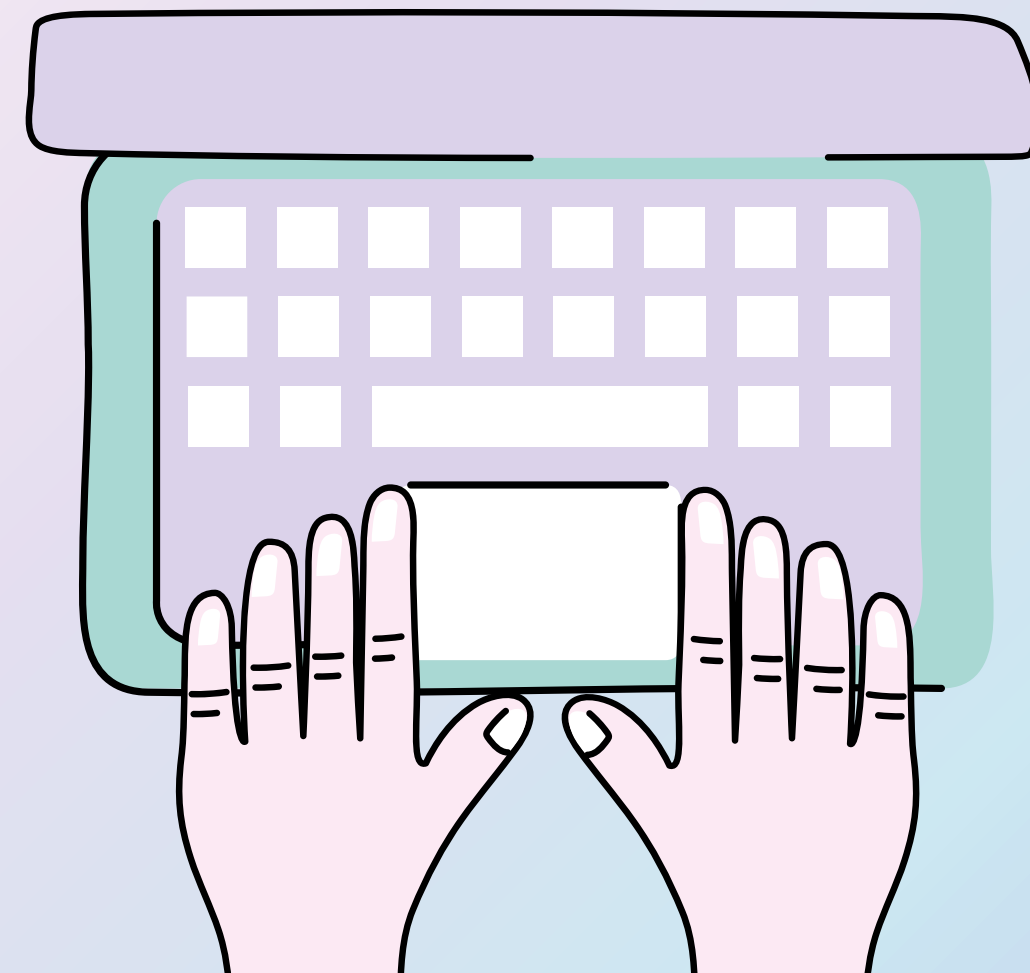
```
void imprimePaciente(Paciente pac) {  
    printf("Dados de Pessoa\n");  
    printf("Nome = %s\n", pac.nome);  
    printf("Idade = %d\n", pac.idade);  
    printf("Altura = %.2f\n", pac.altura);  
    printf("Peso = %.2f\n", pac.peso);  
}
```

```
int main() {  
    Paciente p;  
    strcpy(p.nome, "Joao");  
    p.idade = 28;  
    p.altura = 1.91;  
    p.peso = 88.0;  
    imprimePaciente(p);  
}
```



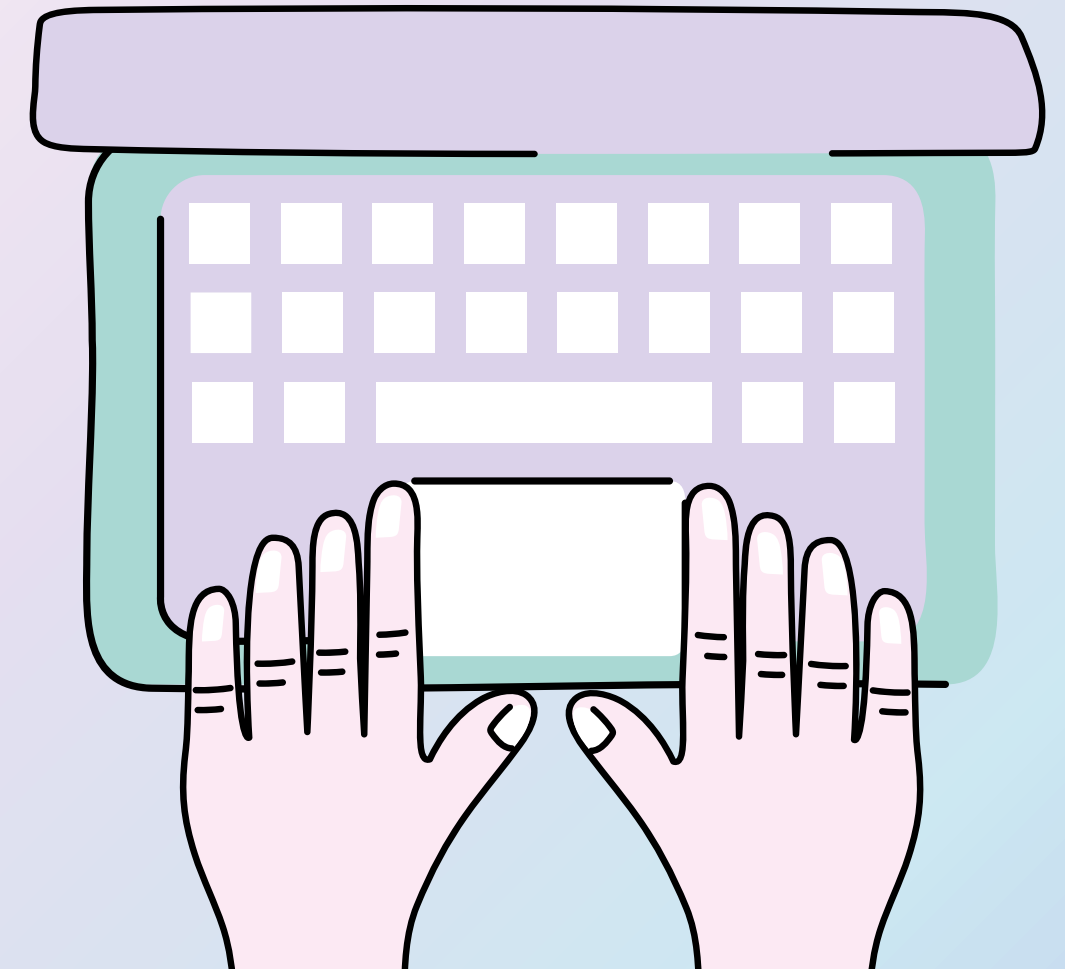
Retornando uma Estrutura

```
Paciente criaPaciente() {  
    Paciente p;  
    printf("Nome paciente: ");  
    scanf("%s", p.nome);  
    printf("Idade paciente: ");  
    scanf("%d", &p.idade);  
    printf("Altura paciente: ");  
    scanf("%f", &p.altura);  
    printf("Peso paciente: ");  
    scanf("%f", &p.peso);  
    return p;  
}  
  
int main() {  
    Paciente p;  
    p = criaPaciente();  
}
```



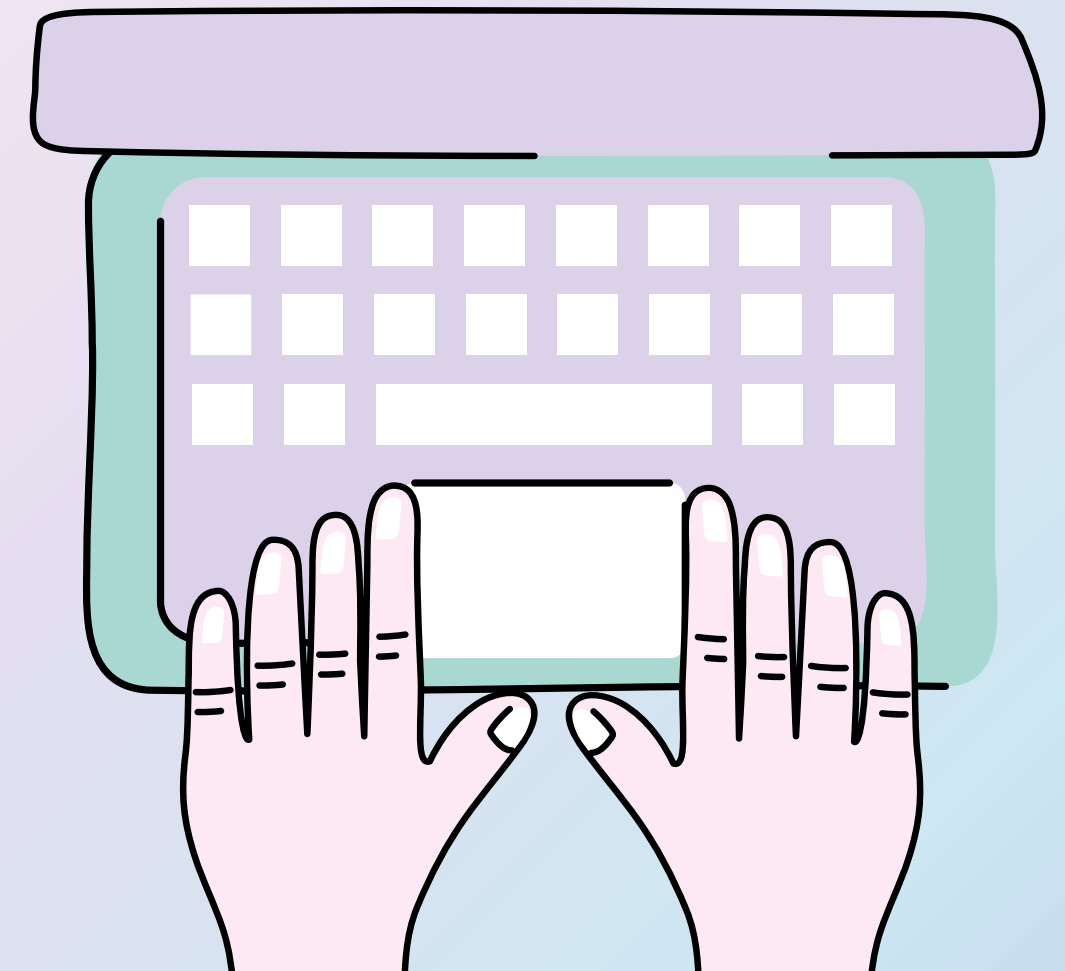
Vetores e Estruturas

```
int main() {  
    int i;  
    Paciente vetor[10];  
    for (i=0; i<10; i++) {  
        printf("Nome paciente: ");  
        scanf("%s", vetor[i].nome);  
        printf("Idade paciente: ");  
        scanf("%d", &vetor[i].idade);  
        printf("Altura paciente: ");  
        scanf("%f", &vetor[i].altura);  
        printf("Peso paciente: ");  
        scanf("%f", &vetor[i].peso);  
    }  
    for (i=0; i<10; i++) {  
        imprimePaciente(vetor[i]);  
    }  
}
```



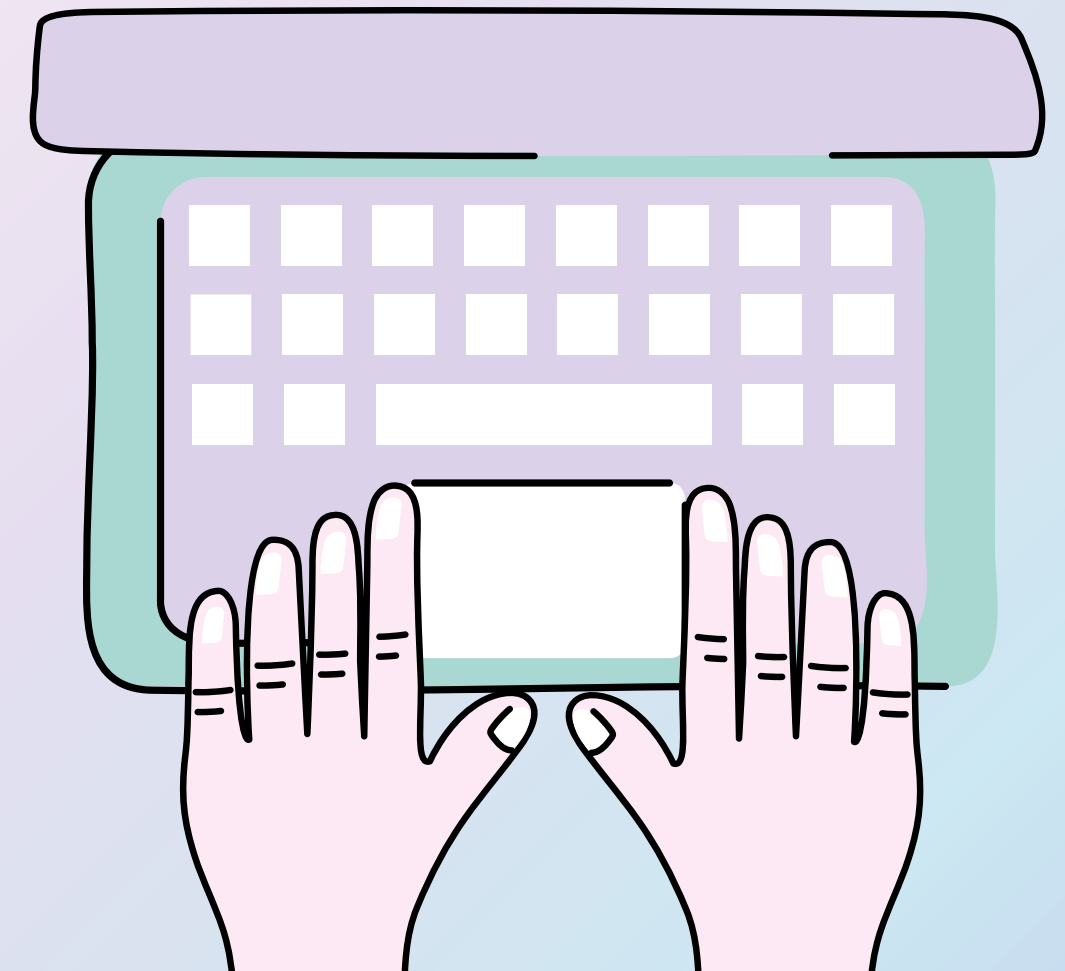
Vetores dentro das Estruturas

```
struct paciente{
    char nome[20];
    int idade;
    float altura, peso;
    char telefones[3][15];
};
typedef struct paciente Paciente;
int main(){
    int i;
    Paciente vetor[2];
    for(i=0;i<2;i++){
        printf("Telefone 1: ");
        scanf("%s", vetor[i].telefones[0]);
        printf("Telefone 2: ");
        scanf("%s", vetor[i].telefones[1]);
        printf("Telefone 3: ");
        scanf("%s", vetor[i].telefones[2]);
    }
}
```



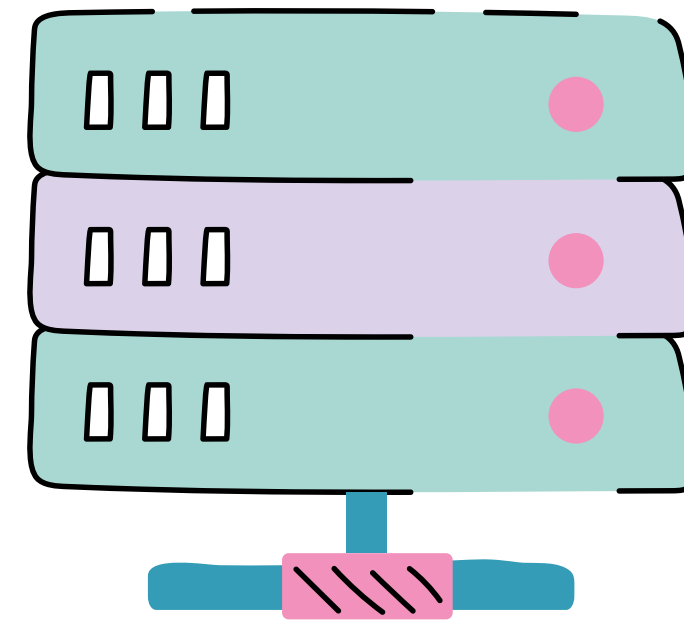
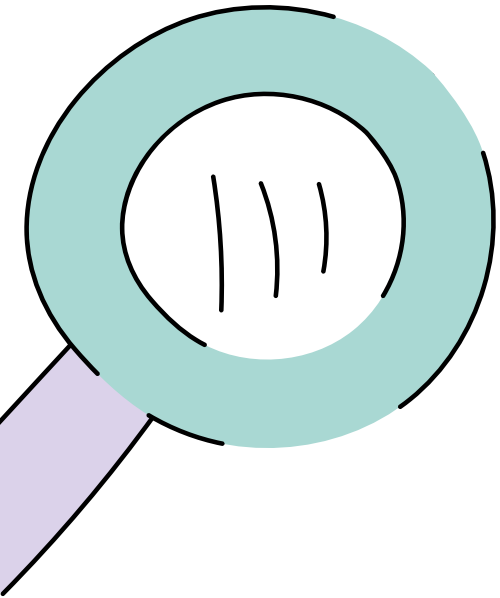
Vetores dentro das Estruturas

```
struct endereco{  
    char cidade[20];  
    char bairro[20];  
    int numero;  
};  
  
typedef struct endereco Endereco;  
  
struct paciente{  
    char nome[20];  
    int idade;  
    float altura, peso;  
    Endereco endereco;  
};  
  
typedef struct paciente Paciente;
```



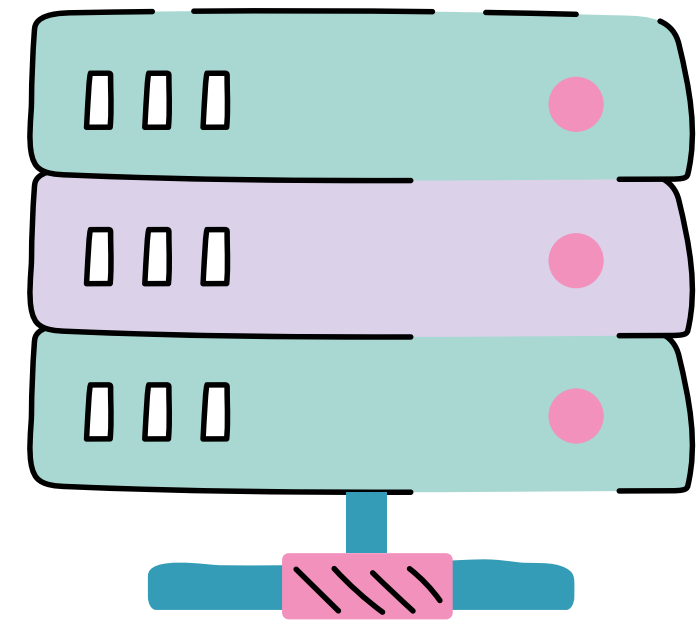
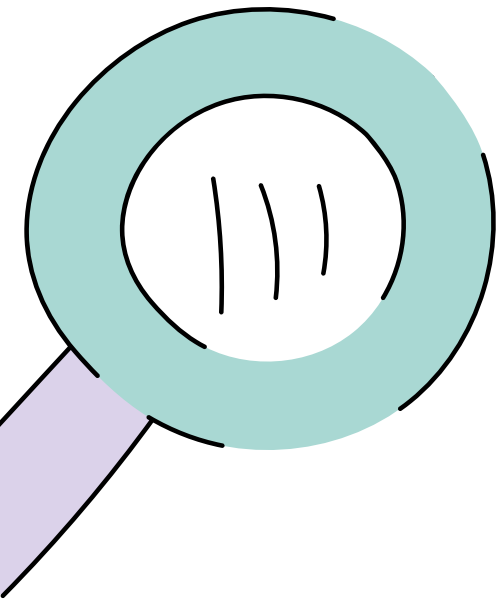
Exercícios

- A partir do exemplo anterior crie uma função que retorne o valor do IMC e diga qual o grau de massa corporal ele se encontra (Abaixo do peso, normal, acima do peso, obesidade...).
- Crie um Tipo Abstrato de Dados (TAD) de uma pessoa, onde possui Nome, CPF, idade. Desenvolva funções para inserir os dados e também para imprimir os dados; No caso de CPF desenvolver uma função que verifica se o valor do CPF está correto; para a idade verifique se ela é maior que 0;



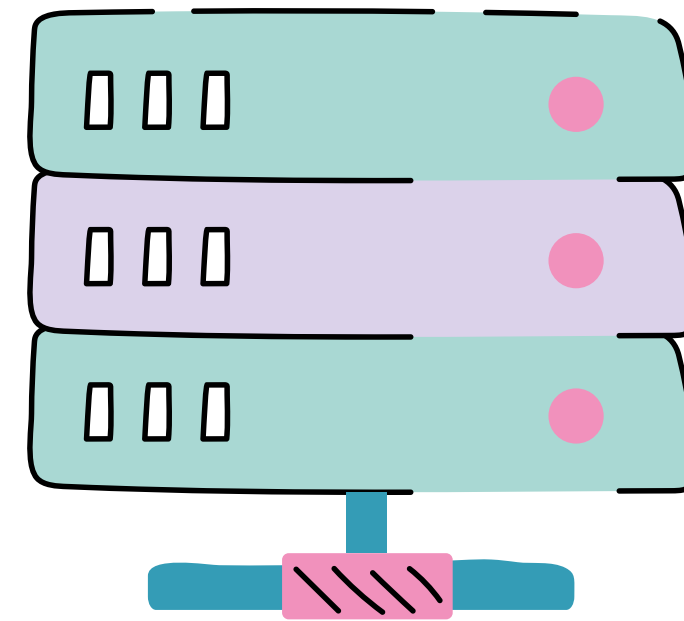
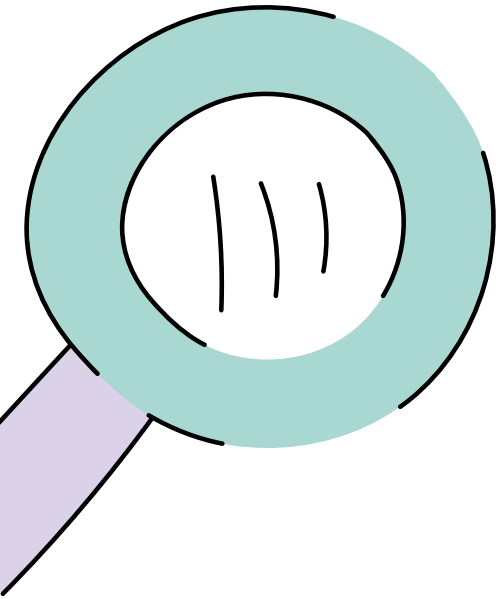
Exercícios

- Defina um Tipo Abstrato de Dados TMatriz, para representar matrizes quadradas de tamanho n . Implemente as operações para somar e multiplicar 2 matrizes. Implemente ainda a operação do cálculo da matriz inversa.



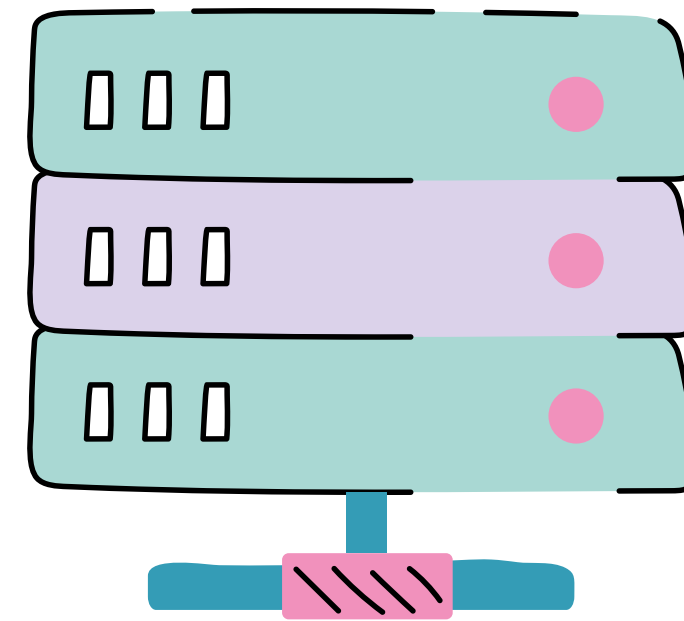
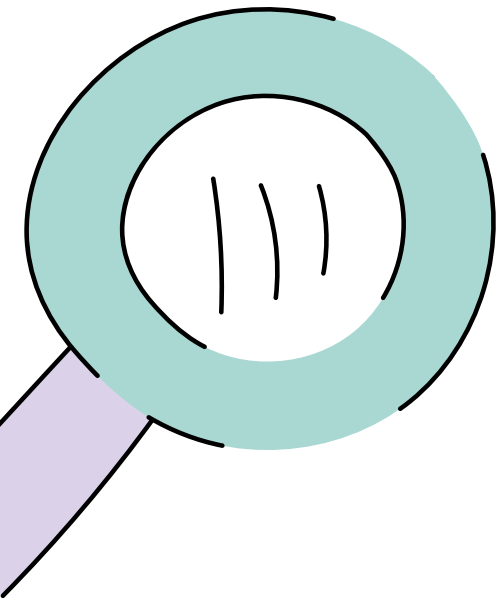
Estrutura de Dados x TADs

- Uma ED é um modo particular de armazenamento e organização de dados em um computador de modo que possam ser usados eficientemente, facilitando sua busca e modificação.
- Então, qual a diferença entre uma estrutura de dados e um tipo abstrato de dados?



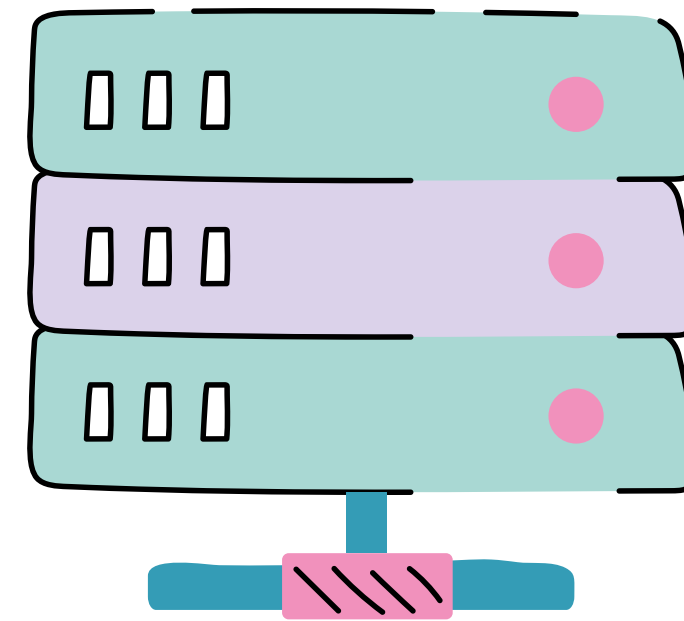
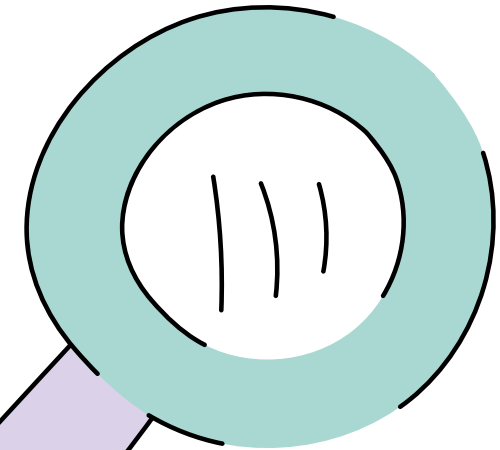
TADs

- Tipo abstrato de dado (TAD) é uma especificação de um conjunto de dados e operações que podem ser executadas sobre esses dados.
- Além disso, é uma metodologia de programação que tem como proposta reduzir a informação necessária para a criação/programação de um algoritmo através da abstração das variáveis envolvidas em uma única entidade fechada, com operações próprias à sua natureza.
- Qualquer processamento a ser realizada sobre os dados encapsulados em um TAD só poderá ser executada por intermédio dos procedimentos definidos no modelo do TAD, sendo esta restrição a característica operacional mais útil dessa estrutura.
- Um programa baseado em TAD deverá conter algoritmos e estruturas de dados que implementem, em termos da linguagem de programação adotada, os procedimentos e os modelos de dados dos TADs utilizados pelo programa.



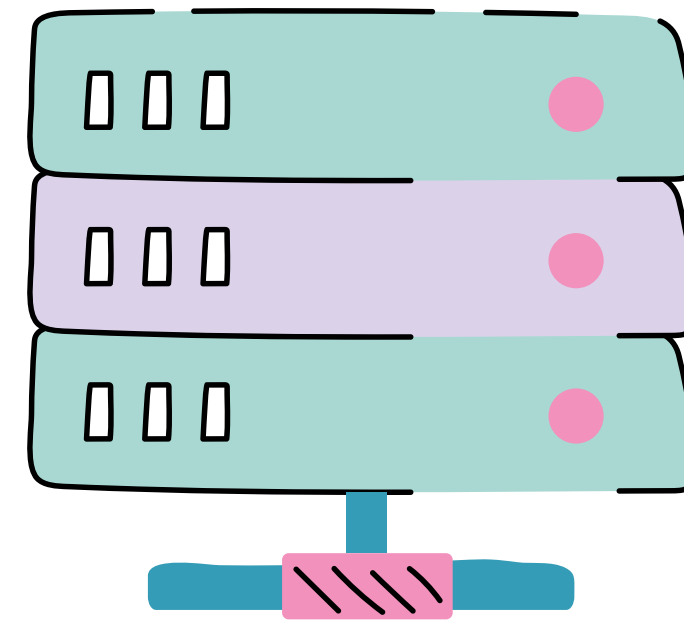
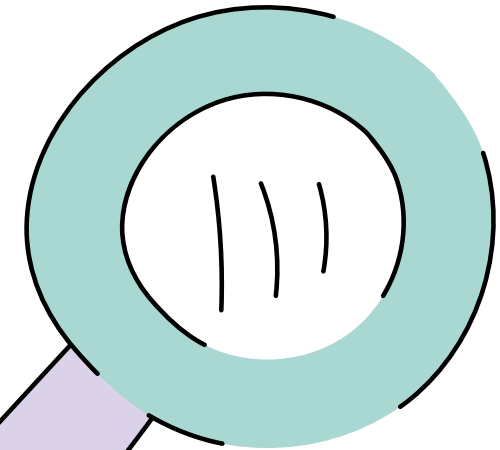
TADs

- Assim, a implementação de cada TAD pode ocupar porções bem definidas do programa: uma para a definição das estruturas de dados e outra para a definição do conjunto de algoritmos.
- Nessas condições, quaisquer alterações realizadas na estrutura de um dado TAD não afetarão as partes do programa que utilizam esse TAD.
- Para definir um TAD, o programador descreve o TAD em dois módulos separados:
 - Um módulo contém a definição do TAD: representação da estrutura de dados e implementação de cada operação suportada
 - Outro módulo contém a interface de acesso: apresenta as operações possíveis;
- Outros programadores podem, por meio da interface de acesso, usar o TAD sem conhecer os detalhes representacionais e sem acessar o módulo de definição.



TADs

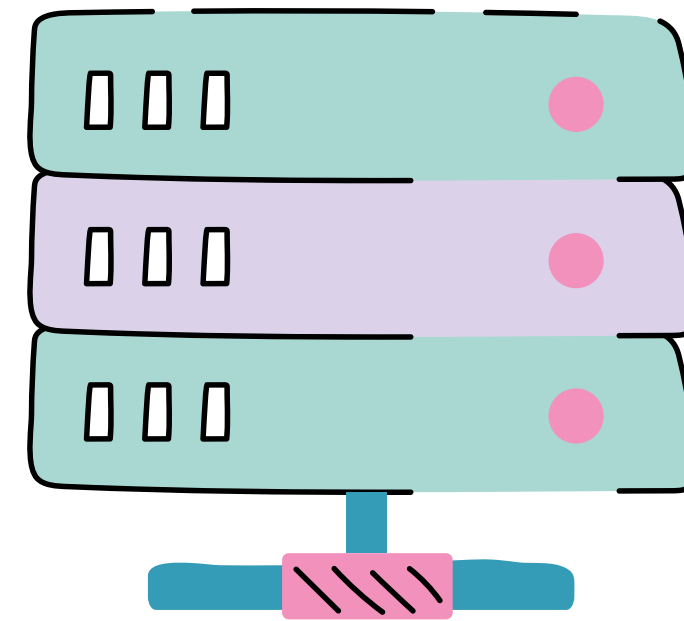
- Uma razão importante para programar em termos de TAD é o fato de que os elementos da estrutura do TAD são acessíveis somente através das operações definidas pelo TAD.
- Essa restrição conduz a uma forma eficiente de programação defensiva, protegendo os dados encapsulados no TAD contra manipulações inesperadas por parte de outros algoritmos.
- Uma segunda razão, também importante, para programar em termos de TAD é o fato de que seu uso permite introduzir alterações nas estruturas definidas no nível de implementação visando, por exemplo, aumento de eficiência livre da preocupação de gerar erros no restante do programa.
- Tais erros não ocorrem porque a única conexão entre o TAD e o restante do programa é aquela constituída pela interface imutável dos algoritmos que implementam a estrutura ativa do TAD.



TADs

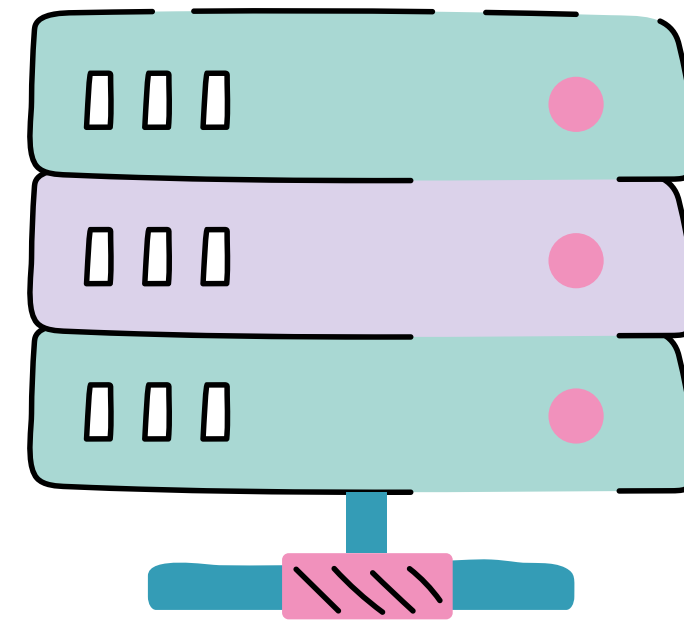
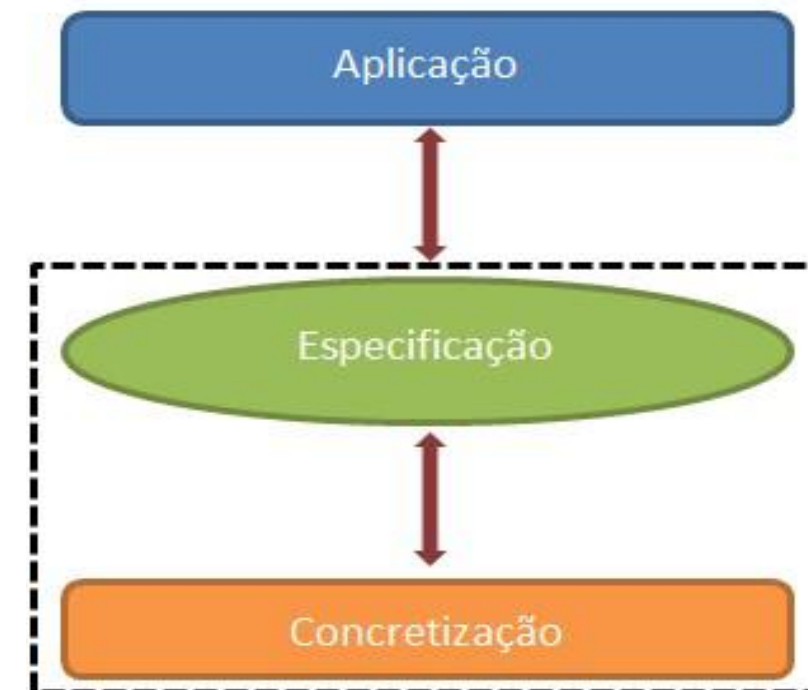
- Por fim, pode-se dizer que um TAD bem construído pode tornar-se uma porção de código confiável e genérica, permitindo e aconselhando seu reuso em outros programas.
- Dessa forma, aumenta-se a produtividade na construção de programas e, sobretudo, garante-se a qualidade dos produtos gerados.
- Exemplo de TAD:

```
estrutura Estudante{  
    Nome  
    Idade  
    Matricula  
}  
  
funcao Estudante_MaiorDeIdade(Estudante estudante) retorna booleano;  
funcao Estudante_ValidaMatricula(Estudante estudante);
```



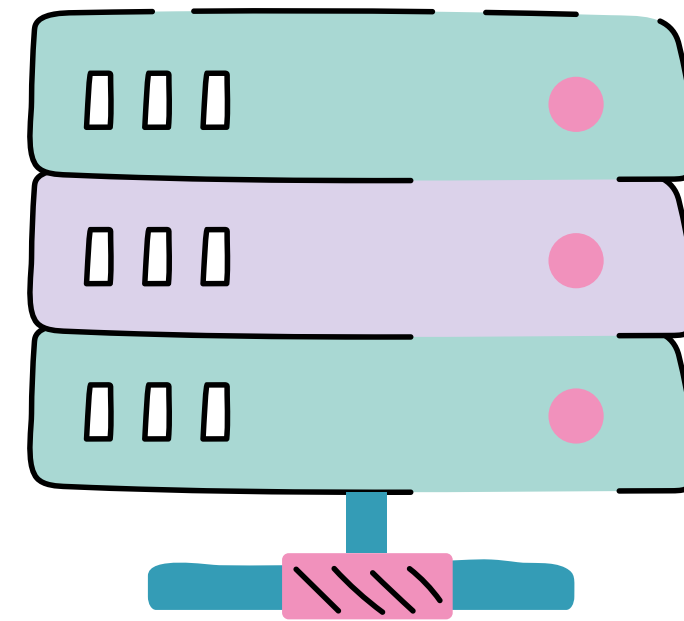
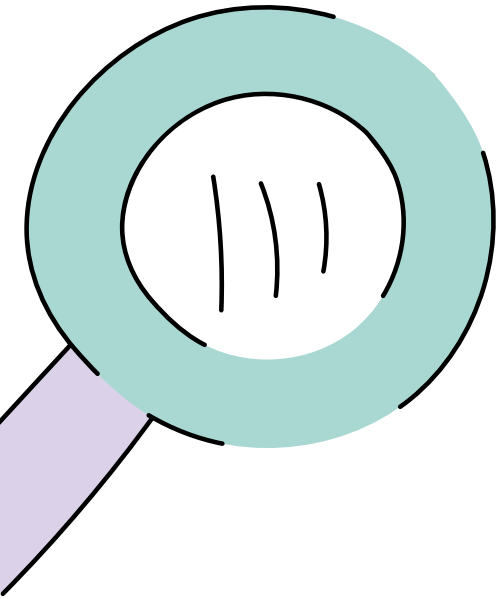
TADs

- Estrutura de um TAD:
- O TAD possui uma especificação, isto é, a definição do que o tipo abstrato representa, e uma implementação, isto é, a criação de um módulo que deve atender a especificação.
- Se esse módulo é encapsulado de forma que nenhum outro dependa de sua implementação, então esse módulo está de acordo com o princípio de *information hiding*.



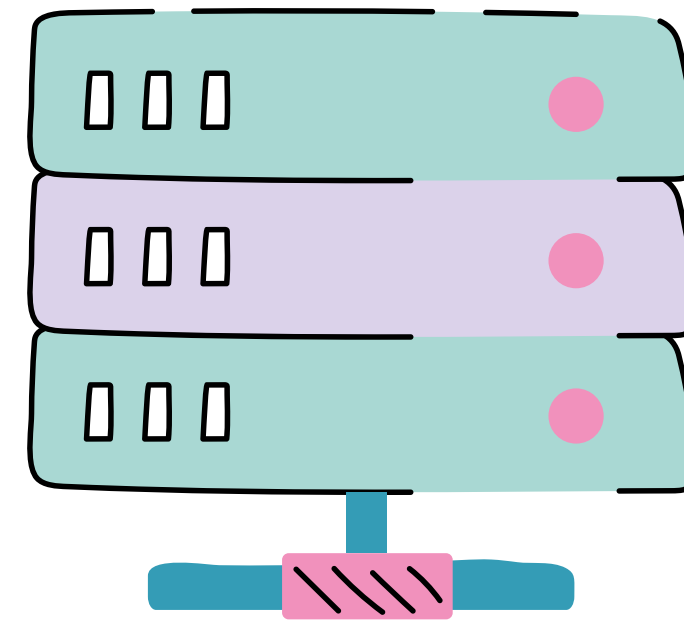
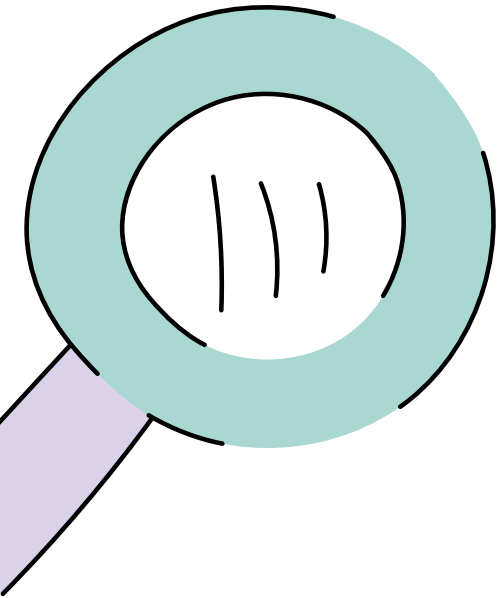
Módulos

- Um módulo na linguagem C pode ser criado utilizando arquivos de cabeçalho e de implementação:
- Arquivo de cabeçalho: Os arquivos '.h' são utilizados para especificar assinatura de funções, definições de constantes, tipos de dados criados pelo usuário etc. De modo geral, os arquivos de cabeçalho tem como função definir a interface de um módulo.
- Arquivo de implementação: Os arquivos '.c' implementam as funções definidas na interface.



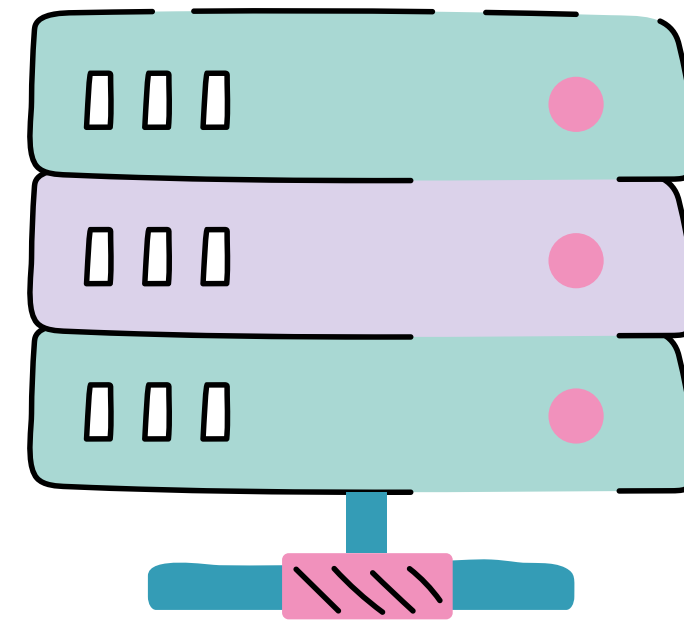
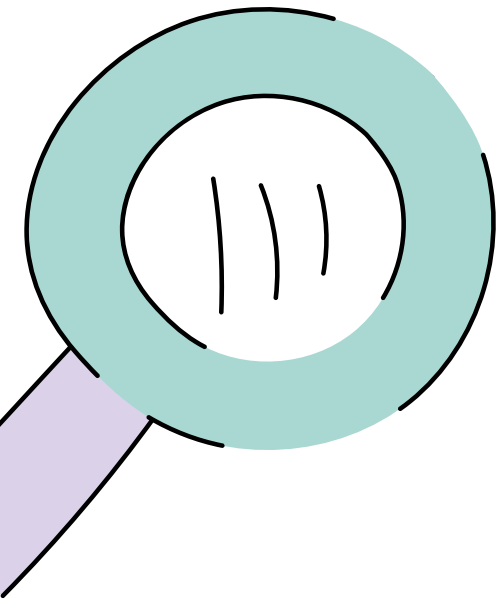
Módulos

- Assim, o arquivo de cabeçalho é utilizado para especificação do tipo de dado abstrato, isto é, das operações que podem ser realizadas e as estruturas de dados definidas.
- Já o arquivo de implementação concretiza as operações definidas na interface.
- Desse modo, uma aplicação pode fazer uso de um determinado módulo a partir da inclusão do seu arquivo de interface.



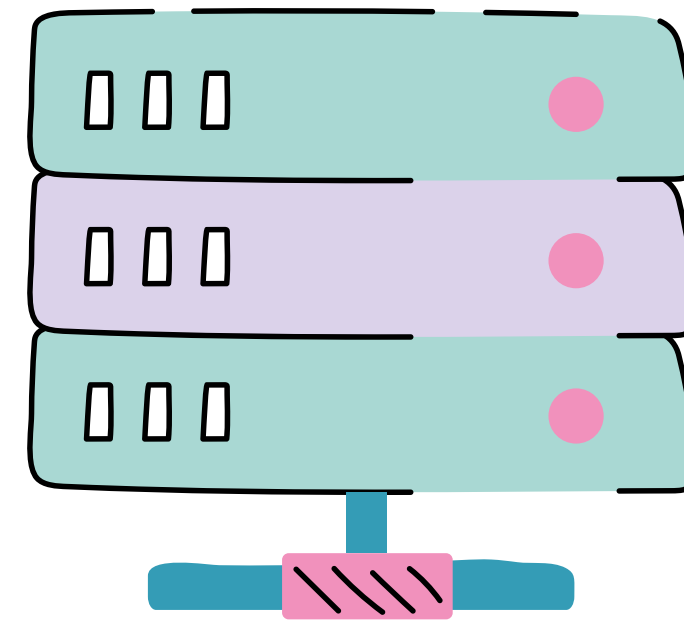
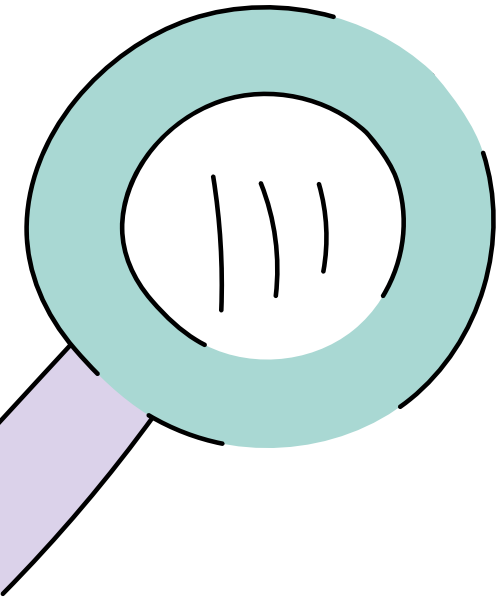
Estruturas Opacas

- Após a apresentação dos conceitos e da visão geral sobre modularização, faremos o uso de estruturas opacas para criar a abstração de dados.
- Lembrando que o conceito envolvido na abstração é separar especificação de implementação, então uma estrutura é opaca se apenas o seu nome é conhecido pela aplicação.
- Isso é possível se declararmos uma estrutura em um arquivo de interface e sua descrição interna em um arquivo de implementação.

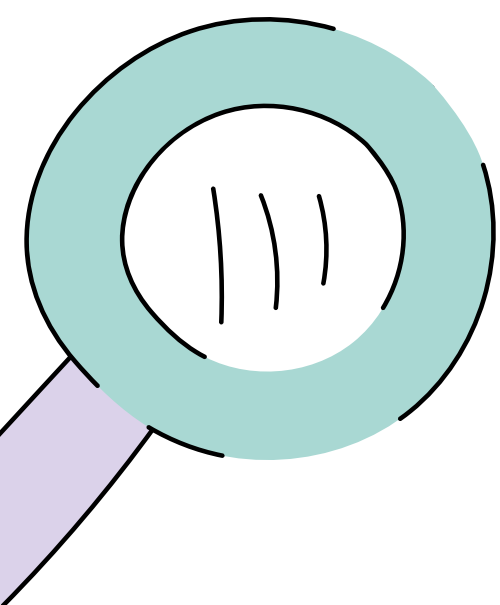


Exemplo

- Vamos definir um TAD Ponto para representar um ponto no R^2 .
- Neste exemplo, vamos considerar as seguintes operações:
 - cria: operação que cria um ponto com coordenadas x e y ;
 - acessaX: operação que devolve a coordenada x de um ponto;
 - acessaY: operação que devolve a coordenada y de um ponto;
 - distancia: operação que calcula a distância entre dois pontos.



Exemplo

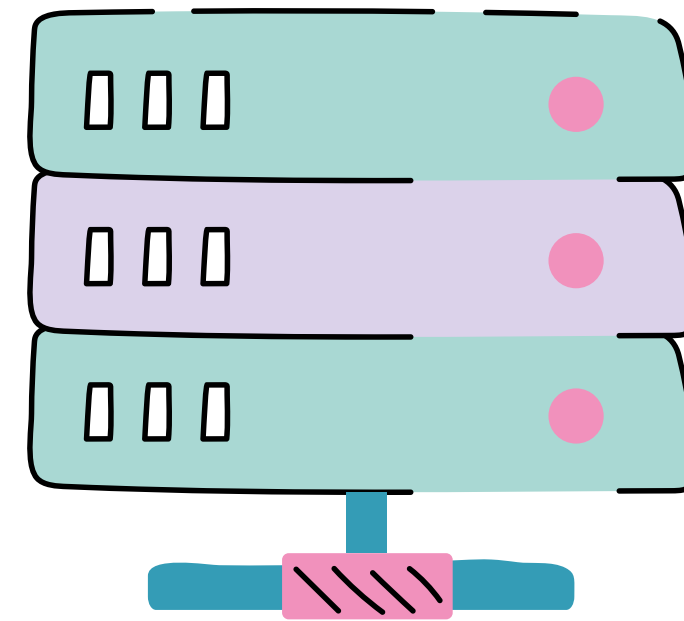


■ Ponto.h

```
#ifndef PONTO_H_INCLUDED
#define PONTO_H_INCLUDED

/* TAD: Ponto (x,y) */
/* Tipo exportado */
typedef struct ponto Ponto;
/* Funções exportadas */
/* Função cria
** Aloca e retorna um ponto com coordenadas (x,y)
*/
Ponto cria (float x, float y);
/* Função acessaX
** Devolve o valor da coordenada x de um ponto
*/
float acessaX (Ponto p);
/* Função acessaY
** Devolve o valor da coordenada y de um ponto
*/
float acessaY (Ponto p);
/* Função distancia
** Retorna a distância entre dois pontos
*/
float distancia (Ponto p1, Ponto p2);

#endif // PONTO_H_INCLUDED
```



Exemplo

- Ponto.c

```
#include <math.h> /* sqrt */
#include "ponto.h"

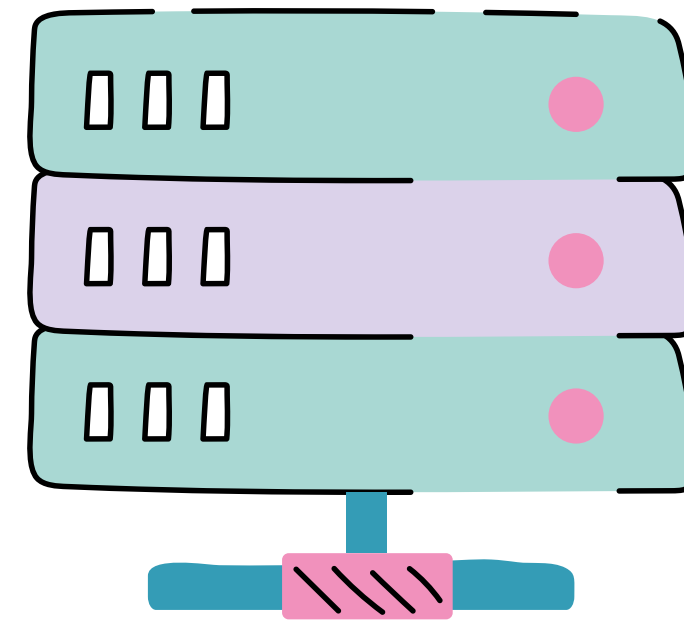
struct ponto {
float x;
float y;
};

Ponto cria (float x, float y) {
Ponto p;
p.x = x;
p.y = y;
return p;
}

float acessaX (Ponto p) {
return p.x;
}

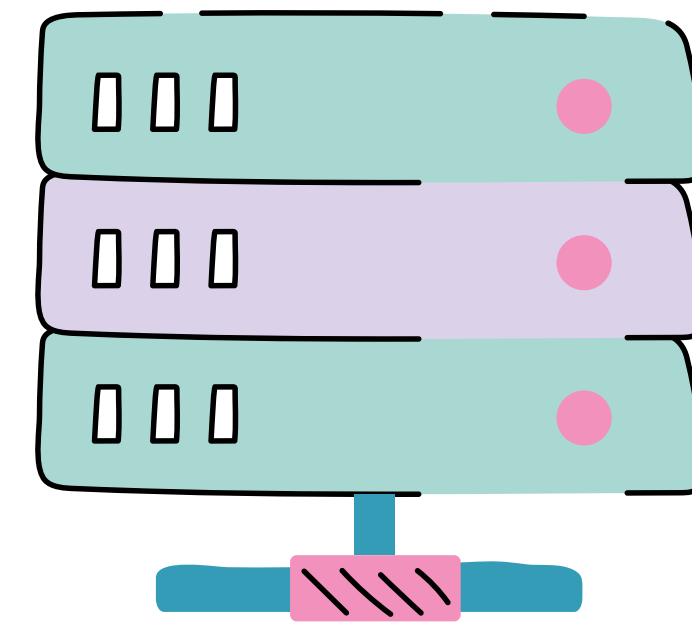
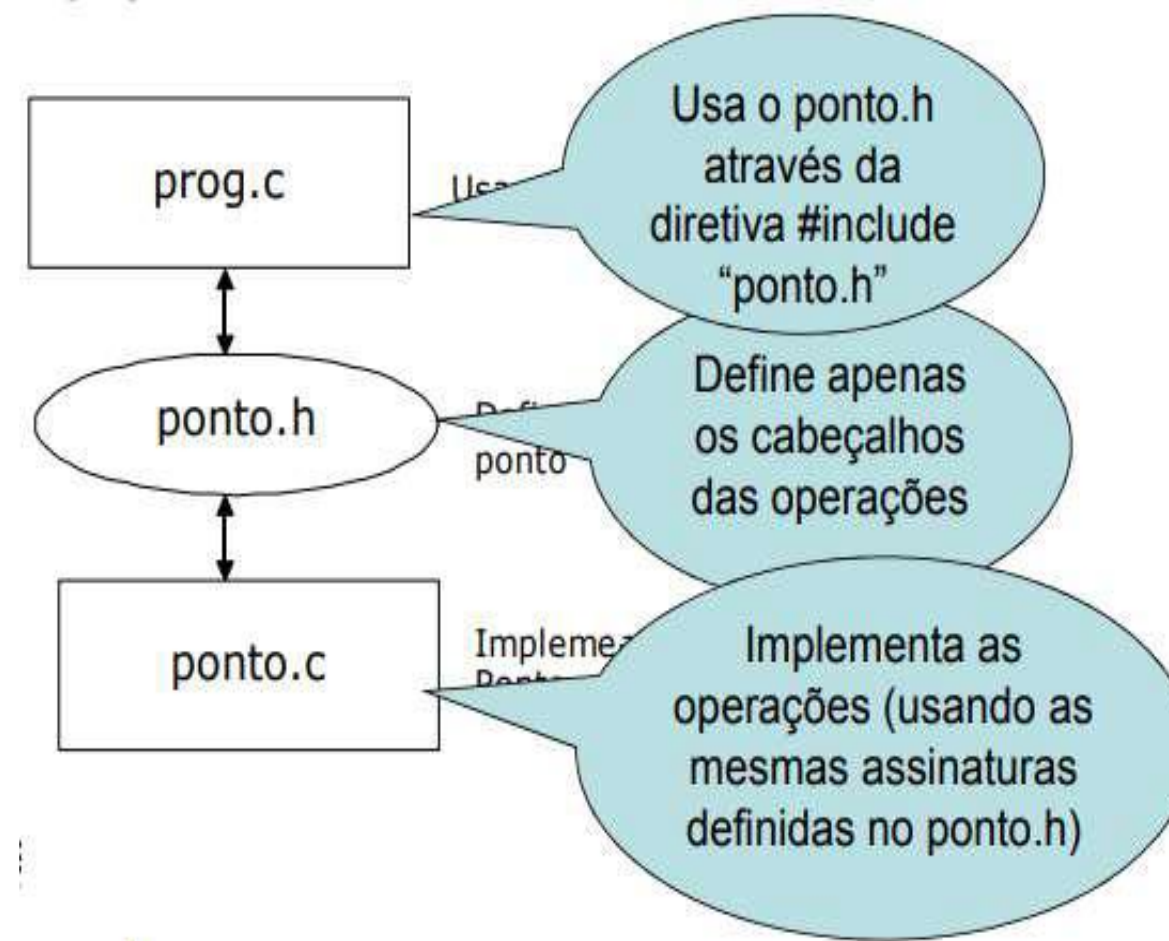
float acessaY (Ponto p) {
return p.y;
}

float distancia (Ponto p1, Ponto p2) {
float dx = p2.x - p1.x;
float dy = p2.y - p1.y;
return sqrt(dx*dx + dy*dy);
}
```



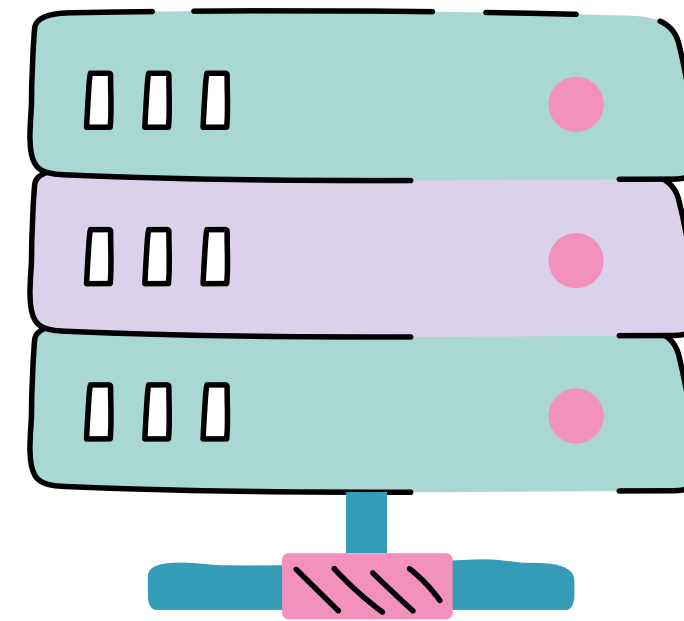
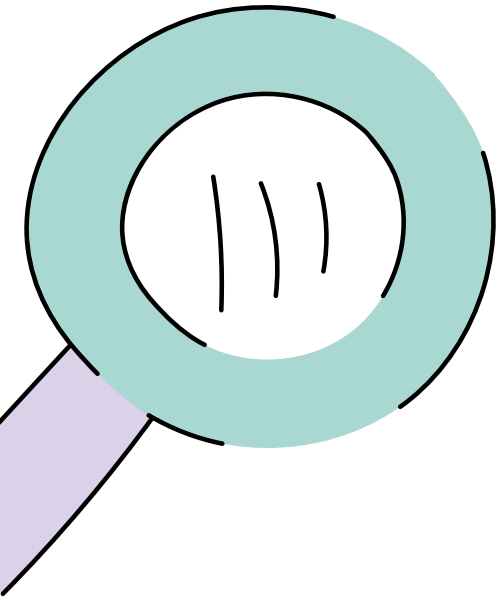
Encapsulamento

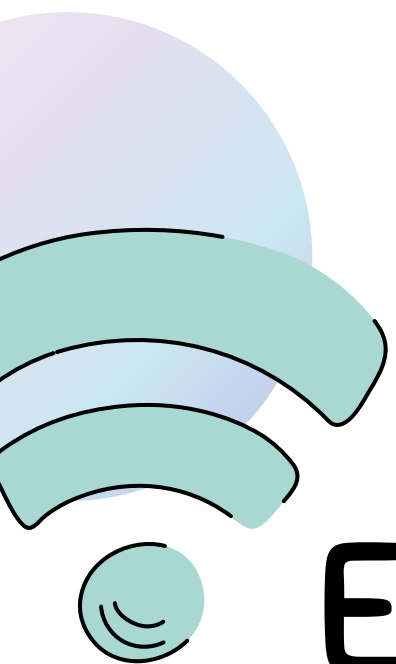
- Compilação
 - `gcc -c ponto.c`
 - `gcc -c prog.c`
- Linkagem
 - `gcc -o prog ponto.o prog.o`
- OU
 - `gcc -o pontos ponto.c prog.c -lm`



Exemplo

- Exercício sobre TADs:
- Implemente um TAD conta bancária com campos número e saldo que suporte as seguintes operações:
 - Iniciar uma nova conta com um número e saldo
 - Depositar um valor na conta
 - Sacar um valor da conta
 - Imprimir o saldo
- Teste o TAD





ESTRUTURA DE DADOS: PONTEIROS

Dra. Maíla de Lima Claro
maila.claro@ifpi.edu.br

